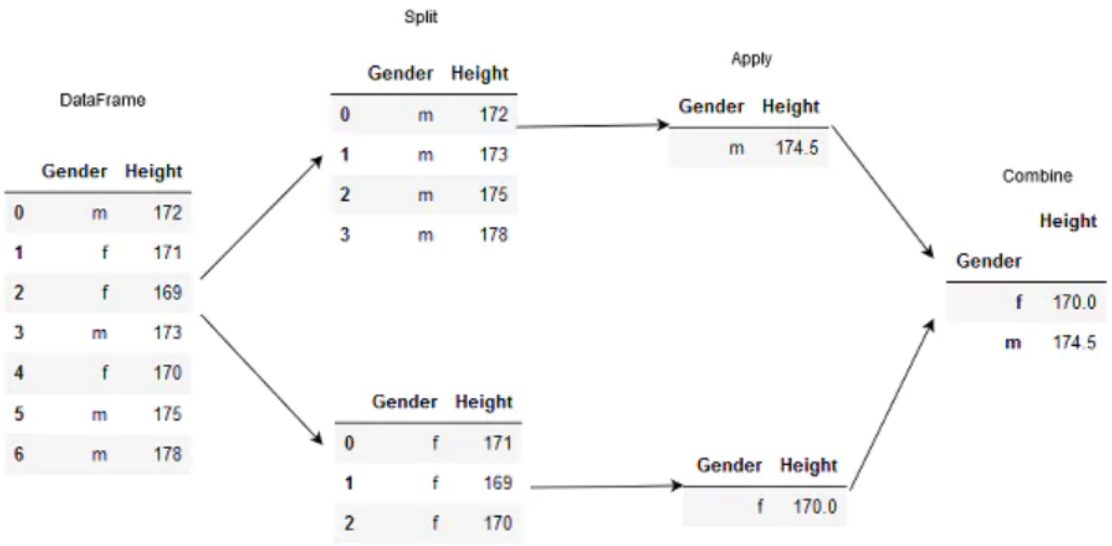
GroupBy

Any groupby operation involves following operations on the original object.

Splitting the Object

Applying a function

Combining the results




```
import pandas as pd
data = {
    'Gender': ['m', 'f', 'f', 'm', 'f', 'm', 'm'],
    'Height': [172, 171, 169, 173, 170, 175, 178]
}
df_sample = pd.DataFrame(data)
df_sample
#Splitting the data into separate groups:
f_filter = df_sample['Gender']=='f'
print(df_sample[f_filter])
m_filter = df_sample['Gender']=='m'
```

```
print(df_sample[m_filter])
#Applying the operation that we need to perform (average in
this case):
f_avg = df_sample[f_filter]['Height'].mean()
m_avg = df_sample[m_filter]['Height'].mean()
print(f_avg,m_avg)


#Combining the result to output a DataFrame
df_output=pd.DataFrame({'Gender':['f','m'],'Height':
[f_avg,m_avg]})
print(df_output)
```

Output:

170.0 174.5

	Gender	*Height*
0	*f*	*170.0*
1	*m*	*174.5*

All these three steps can be achieved by using GroupBy with just a single line of code! Here's how:

```
print(df_sample.groupby('Gender').mean())
```

Output:

	Height

Gender

f   170.0

m   174.5

In many situations, we split the data into sets and we apply some functionality on each subset. In the apply functionality, we can perform the following operations −

Aggregation − computing a summary statistic

Transformation − perform some group-specific operation

Filtration − discarding the data with some condition

Let us now create a DataFrame object

```
#import the pandas library
import pandas as pd
ipl_data = {'Team': ['Riders', 'Riders', 'Devils', 'Devils', 'Kings',
   'kings', 'Kings', 'Kings', 'Riders', 'Royals', 'Royals', 'Riders'],
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],
   'Year':
[2014,2015,2014,2015,2014,2015,2016,2017,2016,2014,2015,2017],
   'Points':[876,789,863,673,741,812,756,788,694,701,804,690]}
df = pd.DataFrame(ipl_data)
print(df)
```

Output:

```
Team  Rank  Year  Points
0   Riders     1  2014     876
1   Riders     2  2015     789
2   Devils     2  2014     863
3   Devils     3  2015     673
4    Kings     3  2014     741
5    kings     4  2015     812
6    Kings     1  2016     756
7    Kings     1  2017     788
8   Riders     2  2016     694
9   Royals     4  2014     701
10  Royals     1  2015     804
11  Riders     2  2017     690
```

Split Data into Groups

Pandas object can be split into any of their objects. There are multiple ways to split an object like –

obj.groupby('key')

obj.groupby(['key1','key2'])

```
# import the pandas library
import pandas as pd
```

```
ipl_data = {'Team': ['Riders', 'Riders', 'Devils', 'Devils', 'Kings',/
                     'kings', 'Kings', 'Kings', 'Riders', 'Royals',/
                     'Royals','Riders'],/
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],/
   'Year': [2014,2015,2014,2015,2014,2015,2016,2017,2016,/
            2014,2015,2017],/
   'Points':[876,789,863,673,741,812,756,788,694,701,804,690]}
df = pd.DataFrame(ipl_data)
print(df.groupby('Team'))
```

Output:

<pandas.core.groupby.generic.DataFrameGroupBy object at 0x00000237101716D0>

View Groups

```
# import the pandas library
import pandas as pd

ipl_data = {'Team':['Riders', 'Riders', 'Devils', 'Devils', 'Kings',/
                    'kings', 'Kings', 'Kings', 'Riders',/
                    'Royals','Royals', 'Riders'],
            'Rank':[1,2,2,3,3,4,1,1,2,4,1,2],
           'Year':[2014,2015,2014,2015,2014,2015,2016,2017,/
                    2016,2014,2015,2017],
          'Points':[876,789,863,673,741,812,756,788,694,701,804,690]}
df = pd.DataFrame(ipl_data)
print(df.groupby('Team').groups)
```

Output:

{'Devils': [2, 3], 'Kings': [4, 6, 7], 'Riders': [0, 1, 8, 11], 'Royals': [9, 10], 'kings': [5]}

Group by with multiple columns

```
# import the pandas library
import pandas as pd

ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',\
                     'Kings','Kings', 'Kings', 'Kings',\
                    'Riders', 'Royals', 'Royals', 'Riders'],
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],/
   'Year': [2014,2015,2014,2015,2014,2015,2016,/
           2017,2016,2014,2015,2017],/
   'Points':[876,789,863,673,741,812,756,788,694,/
            701,804,690]}
df = pd.DataFrame(ipl_data)

print(df.groupby(['Team','Year']).groups)
```

Output:

{('Devils', 2014): [2], ('Devils', 2015): [3], ('Kings', 2014): [4], ('Kings', 2015): [5], ('Kings', 2016): [6], ('Kings', 2017): [7], ('Riders', 2014): [0], ('Riders', 2015): [1], ('Riders', 2016): [8], ('Riders', 2017): [11], ('Royals', 2014): [9], ('Royals', 2015): [10]}

Iterating through Groups

```
# import the pandas library
import pandas as pd

ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',\
                     'Kings','kings', 'Kings', 'Kings',\
                      'Riders', 'Royals', 'Royals','Riders'],\
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],\
   'Year': [2014,2015,2014,2015,2014,2015,2016,\
            2017,2016,2014,2015,2017],
   'Points': [876,789,863,673,741,812,756,788,694,\
            701,804,690]}
df = pd.DataFrame(ipl_data)
grouped = df.groupby('Year')
for year,group in grouped:
   print(year)
   print(group)
```

Output:

2014

```
      Team  Rank  Year  Points
0  Riders     1  2014     876
```

```
2  Devils     2  2014     863
4   Kings     3  2014     741
9  Royals     4  2014     701
2015
      Team  Rank  Year  Points
1   Riders     2  2015     789
3   Devils     3  2015     673
5    kings     4  2015     812
10  Royals     1  2015     804
2016
     Team  Rank  Year  Points
6   Kings     1  2016     756
8  Riders     2  2016     694
2017
      Team  Rank  Year  Points
7    Kings     1  2017     788
11  Riders     2  2017     690
```

Select a Group

Using the **get_group()** method, we can select a single group.

```
# import the pandas library
```

```
import pandas as pd
ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',/
                     'Kings','kings', 'Kings', 'Kings',/
                    'Riders', 'Royals', 'Royals','Riders'],/
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],/
   'Year': [2014,2015,2014,2015,2014,2015,2016,2017,/
            2016,2014,2015,2017],/
   'Points': [876,789,863,673,741,812,756,788,694,/
              701,804,690]}
df = pd.DataFrame(ipl_data)
grouped = df.groupby('Year')
print(grouped.get_group(2014))
```

Output:

	Team	Rank	Year	Points
0	Riders	1	2014	876
2	Devils	2	2014	863
4	Kings	3	2014	741
9	Royals	4	2014	701

Aggregations

An aggregated function returns a single aggregated value for each group. Once the group by object is created, several aggregation operations can be performed on the grouped data. An obvious one is aggregation via the aggregate or equivalent agg method

```
# import the pandas library
```

```
import pandas as pd
import numpy as np

ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',/
                     'Kings','kings', 'Kings', 'Kings',/
                     'Riders', 'Royals', 'Royals', 'Riders'],/
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],/
   'Year': [2014,2015,2014,2015,2014,2015,2016,2017,/
            2016,2014,2015,2017],/
   'Points': [876,789,863,673,741,812,756,788,694,/
              701,804,690]}
df = pd.DataFrame(ipl_data)
grouped = df.groupby('Year')
print(grouped['Points'].agg(np.mean))
```

Output:

```
Year
2014    795.25
2015    769.50
2016    725.00
2017    739.00
Name: Points, dtype: float64
```

Applying Multiple Aggregation Functions at Once With grouped Series, you can also pass a list or dict of functions to do aggregation with, and generate DataFrame as output

```
# import the pandas library
import pandas as pd
import numpy as np

ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',/
                     'Kings','kings', 'Kings', 'Kings',/
                     'Riders', 'Royals', 'Royals','Riders'],/
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],/
   'Year': [2014,2015,2014,2015,2014,2015,2016,2017,/
            2016,2014,2015,2017],/
   'Points': [876,789,863,673,741,812,756,788,/
              694,701,804,690]}
df = pd.DataFrame(ipl_data)
grouped = df.groupby('Team')
print(grouped['Points'].agg([np.sum, np.mean, np.std]))
```

Output:

	sum	mean	std
Team			
Devils	1536	768.000000	134.350288
Kings	2285	761.666667	24.006943
Riders	3049	762.250000	88.567771

```
Royals   1505   752.500000    72.831998
kings     812   812.000000          NaN
```

Named aggregations

In pandas, named aggregations allow you to specify new column names and the aggregation functions to apply to columns when using groupby().agg(). This is especially useful when performing multiple aggregations and wanting to clearly name the resulting columns.

```
import pandas as pd
import numpy as np

ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',\
                     'Kings','kings', 'Kings', 'Kings',\
                     'Riders','Royals','Royals','Riders'],\
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],\
   'Year': [2014,2015,2014,2015,2014,2015,2016,2017,\
            2016,2014,2015,2017],\
   'Points': [876,789,863,673,741,812,756,788,694,\
              701,804,690]}
df = pd.DataFrame(ipl_data)

df.groupby("Team").agg(\
avg_Points = ("Points", "mean"),\
```

```
max_Points = ("Points", "max")\
)
```

Output:

	avg_Points	max_Points
Team		
Devils	768.000000	863
Kings	761.666667	788
Riders	762.250000	876
Royals	752.500000	804
kings	812.000000	812

Sorting the results

We can sort the output based on the aggregated columns using the sort_values function.

```
import pandas as pd
import numpy as np

ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',\
                     'Kings','kings', 'Kings', 'Kings',\
                     'Riders', 'Royals', 'Royals','Riders'],\
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],\
   'Year': [2014,2015,2014,2015,2014,2015,2016,2017,\
            2016,2014,2015,2017],\
   'Points': [876,789,863,673,741,812,756,788,694,\
```

```
                701,804,690]}
df = pd.DataFrame(ipl_data)
df1=df.groupby("Team", as_index=False).agg(\
avg_points = ("Points", "mean")\
).sort_values(by="avg_points", ascending=False).head()
print(df1)
```

Output:

```
     Team  avg_points
4   kings  812.000000
0  Devils  768.000000
2  Riders  762.250000
1   Kings  761.666667
3  Royals  752.500000
```

Plot the results

To **group** your data by one or more columns (e.g. categories, data) and then **visualize** the results using seaborn

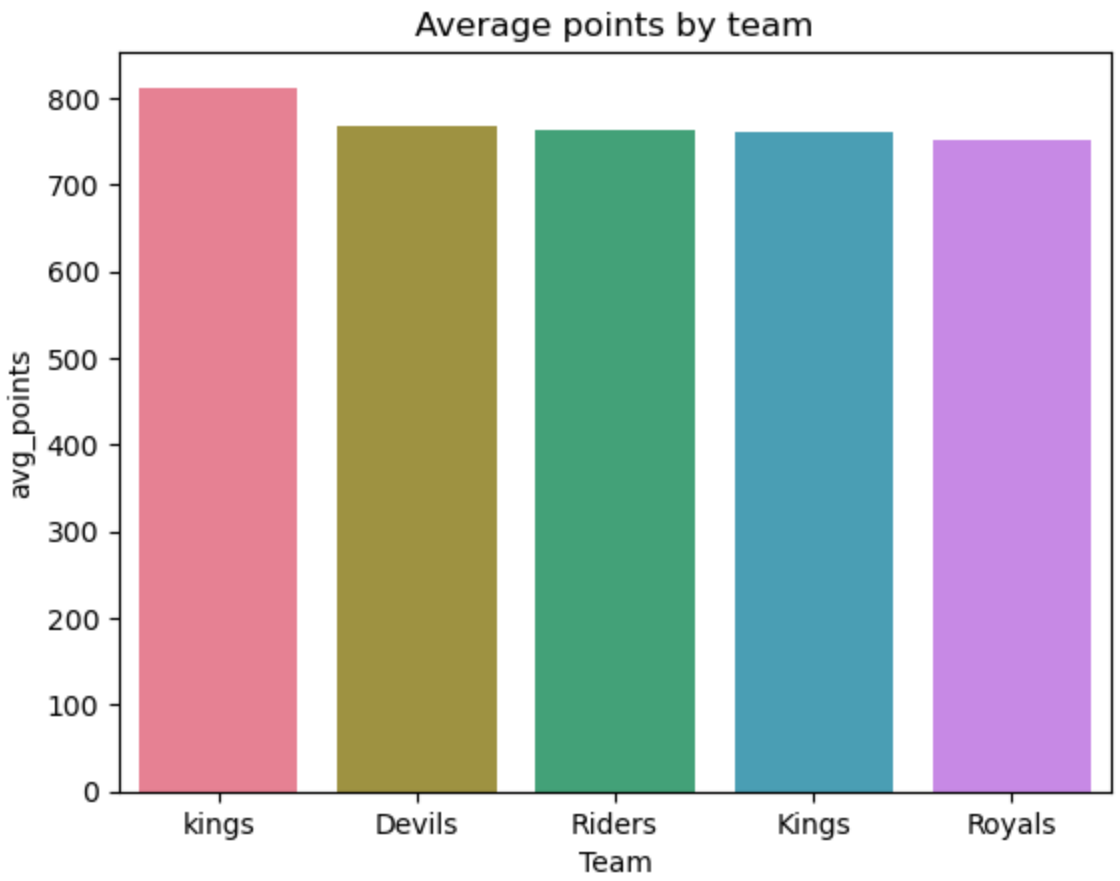
```
import pandas as pd
import numpy as np

ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',\
                     'Kings','kings', 'Kings', 'Kings', \
```

```
                    'Riders', 'Royals', 'Royals','Riders'],\
    'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],\
    'Year': [2014,2015,2014,2015,2014,2015,2016,2017,\
             2016,2014,2015,2017],\
    'Points': [876,789,863,673,741,812,756,788,694,\
              701,804,690]}
df = pd.DataFrame(ipl_data)
df1=df.groupby("Team", as_index=False).agg(
avg_points = ("Points", "mean")
).sort_values(by="avg_points", ascending=False).head()
print(df1)
import seaborn as sns
import matplotlib.pyplot as plt
sns.barplot(x='Team',y='avg_points',data=df1,palette='husl')
plt.title('Average points by team')
plt.show()
```

Output:

```
     Team  avg_points
4   kings  812.000000
0  Devils  768.000000
2  Riders  762.250000
1   Kings  761.666667
3  Royals  752.500000
```



Identify the unique values in column

The unique function can be used for finding the unique values in each group. For instance, we can find the unique product codes in each group as follows:

```
import pandas as pd
import numpy as np

ipl_data = {'Team': ['Riders', 'Riders', 'Devils', 'Devils',
                     'Kings','kings', 'Kings', 'Kings', \
                     'Riders','Royals','Royals','Riders'],\
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],\
   'Year':[2014,2015,2014,2015,2014,2015,2016,2017,\
```

```
            2016,2014,2015,2017],\
   'Points': [876,789,863,673,741,812,756,788,694,\
              701,804,690]}
df = pd.DataFrame(ipl_data)

df1=df.groupby("Team", as_index=False).agg(\
unique_values = ("Year","unique")
)
print(df1)
```

Output:

```
Team unique_values
0    Devils    [2014, 2015]
1    Kings     [2014, 2016, 2017]
2    Riders    [2014, 2015, 2016, 2017]
3    Royals    [2014, 2015]
4    kings     [2015]
```

Number of unique values

We can also find the number of unique values in each group using the nunique function.

```
import pandas as pd
import numpy as np
```

```
ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',\
                     'Kings','kings', 'Kings', 'Kings', \
                     'Riders','Royals','Royals','Riders'],\
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],\
   'Year':[2014,2015,2014,2015,2014,2015,2016,2017,\
           2016,2014,2015,2017],\
   'Points':[876,789,863,673,741,812,756,788,694,701,\
            804,690]}
df = pd.DataFrame(ipl_data)

df1=df.groupby("Team", as_index=False).agg(\
number_of_unique_values = ("Year","nunique")\
)
print(df1)
```

Output:

	Team	*number_of_unique_values*
0	*Devils*	*2*
1	*Kings*	*3*
2	*Riders*	*4*
3	*Royals*	*2*
4	*kings*	*1*

Lambda expressions

We can use lambda expressions as aggregations in the agg function.

```
import pandas as pd
import numpy as np
ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',\
                     'Kings','kings', 'Kings', 'Kings',\
                     'Riders','Royals','Royals','Riders'],\
   'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],\
   'Year': [2014,2015,2014,2015,2014,2015,2016,2017,\
            2016,2014,2015,2017],\
   'Points':[876,789,863,673,741,812,756,788,694,\
            701,804,690]}
df = pd.DataFrame(ipl_data)
df1=df.groupby("Team").agg(total_points_in_hundreds = (\
"Points", lambda x: round(x.sum() / 100, 1)))
print(df1)
```

Output:

total_points_in_hundreds

Team

Devils *15.4*

Kings *22.8*

Riders *30.5*

Royals *15.0*

kings *8.1*

Transformations

Transformation on a group or a column returns an object that is indexed the same size of that is being grouped. Thus, the transform should return a result that is the same size as that of a group chunk. Transformation can be done only with one column, to work with multiple columns use apply() function.

```
import pandas as pd
import numpy as np

# Create the DataFrame
ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',\
                     'Kings','Kings', 'Kings', 'Kings',\
                     'Riders','Royals','Royals','Riders'],\
            'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],\
            'Year': [2014,2015,2014,2015,2014,2015,2016,\
                     2017, 2016, 2014, 2015, 2017],\
            'Points': [876, 789, 863, 673, 741, 812, 756,\
                       788, 694, 701, 804, 690]}
df = pd.DataFrame(ipl_data)

# Group by 'Team' and normalize 'Points' within each group
df['NormalizedPoints'] =
\df.groupby('Team')['Points'].transform(\
lambda x: (x - x.mean()) / x.std())
print(df)
```

Output:

	Team	Rank	Year	Points	NormalizedPoints
0	Riders	1	2014	876	1.284327
1	Riders	2	2015	789	0.302029
2	Devils	2	2014	863	0.707107
3	Devils	3	2015	673	-0.707107
4	Kings	3	2014	741	-1.042333
5	Kings	4	2015	812	1.183401
6	Kings	1	2016	756	-0.572108
7	Kings	1	2017	788	0.431040
8	Riders	2	2016	694	-0.770596
9	Royals	4	2014	701	-0.707107
10	Royals	1	2015	804	0.707107
11	Riders	2	2017	690	-0.815759

Custom transformation

Custom functions can be applied to each elements in the row.

```
import pandas as pd
import numpy as np
```

```
# Create the DataFrame
ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',\
                     'Kings','Kings', 'Kings', 'Kings',\
                     'Riders','Royals','Royals','Riders'],\
            'Rank': [1,2,2,3,3,4,1,1,2,4,1,2],\
            'Year': [2014,2015,2014,2015,2014,2015,2016,\
                     2017, 2016, 2014, 2015, 2017],\
            'Points': [876,789,863,673,741,812,756,788,\
                       694, 701, 804, 690]}
df = pd.DataFrame(ipl_data)
def custom_transform(x):
    return x - x.max()

# Apply custom transformation to the 'Points' column within
each 'Team'
df['PointsDifferenceFromMax']=\
df.groupby('Team')['Points'].transform(custom_transform)
print('mean',df['Points'].max())
print(df)
```

Output:

mean: 876

	Team	Rank	Year	Points	PointsDifferenceFromMax
0	Riders	1	2014	876	0
1	Riders	2	2015	789	-87
2	Devils	2	2014	863	0
3	Devils	3	2015	673	-190

```
4    Kings     3  2014     741                    -71
5    Kings     4  2015     812                      0
6    Kings     1  2016     756                    -56
7    Kings     1  2017     788                    -24
8   Riders     2  2016     694                   -182
9   Royals     4  2014     701                   -103
10  Royals     1  2015     804                      0
11  Riders     2  2017     690                   -186
```

Filtration

Filtration filters the data on a defined criteria and returns the subset of data. The filter() function is used to filter the data.

```
import pandas as pd
import numpy as np
ipl_data = {'Team': ['Riders', 'Riders', 'Devils','Devils',\
                     'Kings','kings', 'Kings', 'Kings', \
                     'Riders', 'Royals','Royals', 'Riders'],\
   'Rank': [1,2, 2,3,3,4,1,1,2,4,1,2],\
   'Year': [2014,2015,2014,2015,2014,2015,2016,2017,\
           2016,2014,2015,2017],\
   'Points':[876,789,863,673,741,812,756,788,694,701,\
             804,690]}
df = pd.DataFrame(ipl_data)
print(df.groupby('Team').filter(lambda x: len(x) >= 3))
```

Output:

	Team	Rank	Year	Points
0	Riders	1	2014	876
1	Riders	2	2015	789
4	Kings	3	2014	741
6	Kings	1	2016	756
7	Kings	1	2017	788
8	Riders	2	2016	694
11	Riders	2	2017	690